

Practical exercise (Google BigQuery)

1.

```
3 --- Q1. Filter all transactions that occurred in the year 2023. Expected output: All columns
4 SELECT *
5 FROM `retail-478310.Sales.Data`
6 WHERE EXTRACT(YEAR FROM `Date`) = 2023;
7
```

Query completed

Using on-demand processing quota

Query results

Job information	Results	Visualisation	JSON	Execution details	Execution graph				
Row	Transaction ID	Date	Customer ID	Gender	Age	Product Category	Quantity	Price per Unit	Total Amount
1	191	2023-10-18	CUST191	Male	64	Beauty	1	25	25
2	204	2023-09-28	CUST204	Male	39	Beauty	1	25	25
3	230	2023-04-23	CUST230	Male	54	Beauty	1	25	25
4	232	2023-02-06	CUST232	Female	43	Beauty	1	25	25
5	309	2023-12-23	CUST309	Female	26	Beauty	1	25	25

2.

```
8 --- Q2. Display all transactions where the Total Amount is more than the average Total Amount of the entire dataset. Expected output: All columns
9 SELECT *
10 FROM `retail-478310.Sales.Data`
11 WHERE `Total Amount` > (SELECT AVG(`Total Amount`) FROM `retail-478310.Sales.Data`);
12
```

Query completed

Using on-demand processing quota

Query results

Job information	Results	Visualisation	JSON	Execution details	Execution graph				
Row	Transaction ID	Date	Customer ID	Gender	Age	Product Category	Quantity	Price per Unit	Total Amount
1	21	2023-01-14	CUST021	Female	50	Beauty	1	500	500
2	28	2023-04-23	CUST028	Female	43	Beauty	1	500	500

3.

```
13 --- Q3. Calculate the total revenue (sum of Total Amount). Expected output: Total_Revenue
14 SELECT SUM(`Total Amount`) AS Total_Revenue
15 FROM `retail-478310.Sales.Data`;
16
17
18
19
20
21
22
23
24
25
26
27
28
```

Query completed

Using on-demand processing quota

Query results

Job information	Results	Visualisation	JSON	Execution details	Execution graph
Row	Total_Revenue				
1	456000				

```

17 --- Q4. Display all distinct Product Categories in the dataset. Expected output: Product_Category
18 SELECT DISTINCT `Product Category`
19 FROM `retail-478310.Sales.Data`;
20
21
22
23
24
25
26

```

✓ Query completed

Using on-demand processing quota

← Query results

Job information **Results** Visualisation JSON Execution details Execution graph

Row	Product Category ▾	
1	Beauty	
2	Clothing	
3	Electronics	

4.

```

21 --- 5. GROUP BY Q5. For each Product Category, calculate the total quantity sold. Expected output: Product_Category, Total_Quantity
22 SELECT `Product Category`, SUM(Quantity) AS Total_Quantity
23 FROM `retail-478310.Sales.Data`
24 GROUP BY `Product Category`;
25
26
27
28
29
30
31
32
33
34
35
36

```

✓ This script will process 254.03 KB when run.

Using on-demand processing quota

← Query results

Job information **Results** Visualisation JSON Execution details Execution graph

Row	Product Category ▾	Total_Quantity ▾
1	Beauty	771
2	Clothing	894
3	Electronics	849

5.

```

26 --- Q6. Create a column called Age_Group that classifies customers as 'Youth' (<30), 'Adult' (30-59), and 'Senior' (60+). Expected output: Customer_ID, Age, Age_Group
27 SELECT 'Customer_ID', Age,
28 CASE
29 WHEN Age < 30 THEN 'Youth'
30 WHEN Age BETWEEN 30 AND 59 THEN 'Adult'
31 WHEN Age >= 60 THEN 'Senior'
32 END AS Age_Group
33 FROM `retail-478310.Sales.Data`;
34
35
36

```

Query completed

Using on-demand processing quota

Query results

Job information	Results	Visualisation	JSON	Execution details	Execution graph
Row	Customer ID	Age	Age_Group		
1	CUST191	64	Senior		
2	CUST204	39	Adult		
3	CUST230	54	Adult		
4	CUST232	43	Adult		
5	CUST309	26	Youth		

6.

```

35 --- Q7. For each Gender, count how many high-value transactions occurred (where Total Amount > 500). Expected output: Gender, High_Value_Transaction
36 SELECT
37 Gender,
38 COUNT(DISTINCT `Transaction ID`) AS High_Value_Transactions
39 FROM
40 `retail-478310`.`Sales`.`Data`
41 WHERE
42 `Total Amount` > 500
43 GROUP BY
44 Gender;
45
46
47
48
49
50

```

Query completed

Using on-demand processing quota

Query results

Job information	Results	Visualisation	JSON	Execution details	Execution graph
Row	Customer ID	Age	Age_Group		
1	CUST191	64	Senior		
2	CUST204	39	Adult		
3	CUST230	54	Adult		
4	CUST232	43	Adult		
5	CUST309	26	Youth		

7.

```

47 --- Q8. For each Product Category, show only those categories where the total revenue exceeds 5,000. Expected output: Product_Category, Total_Revenue
48 SELECT `Product Category`, SUM(`Total Amount`) AS Total_Revenue
49 FROM `retail-478310`.`Sales`.`Data`
50 GROUP BY `Product Category`
51 HAVING SUM(`Total Amount`) > 5000;
52
53
54
55
56
57
58
59
60
61
62

```

Query completed

Using on-demand processing quota

Query results

Job information	Results	Visualisation	JSON	Execution details	Execution graph
Row	Product Category	Total_Revenue			
1	Beauty	143515			
2	Clothing	155580			
3	Electronics	156905			

8.

9.

```

52 --- Q9. Display a new column called Unit_Cost_Category that labels a transaction as: - 'Cheap' if Price per Unit < 50 - 'Moderate' if Price per Unit between 50 and 200 - 'Expensive' if Price per Unit > 200 Expected output: Transaction_ID,
53 Price_per_Unit, Unit_Cost_Category
54 SELECT
55   'Transaction ID',
56   'Price per Unit',
57   CASE
58     WHEN 'Price per Unit' < 50 THEN 'Cheap'
59     WHEN 'Price per Unit' BETWEEN 50 AND 200 THEN 'Moderate'
60     WHEN 'Price per Unit' > 200 THEN 'Expensive'
61   END AS Unit_Cost_Category
62 FROM 'retail-478316'.Sales'.Data';

```

Query completed

Using on-demand processing quota

Query results

Save results Open in

Job information	Results	Visualisation	JSON	Execution details	Execution graph
Row	Transaction ID	Price per Unit	Unit_Cost_Category		
1	191	25	Cheap		
2	204	25	Cheap		
3	230	25	Cheap		
4	232	25	Cheap		
5	309	25	Cheap		
6	310	25	Cheap		
7	363	25	Cheap		

```

63 --- Q10. Display all transactions from customers aged 40 or older and add a column Spending_Level showing 'High' if Total Amount > 1000, otherwise 'Low'. Expected output: Customer_ID, Age, Total_Amount, Spending_Level
64 SELECT
65   'Customer ID',
66   'Age',
67   'Total Amount',
68   CASE WHEN 'Total Amount' > 1000 THEN 'High' ELSE 'Low'
69   END AS Spending_Level
70 FROM 'retail-478316'.Sales'.Data'
71 WHERE Age >= 40;
72
73
74

```

This script will process 351.13 KB when run.

Using on-demand processing quota

Query results

Save results

Job information	Results	Visualisation	JSON	Execution details	Execution graph
Row	Customer ID	Age	Total Amount	Spending_Level	
1	CUST191	64	25	Low	
2	CUST230	54	25	Low	
3	CUST232	43	25	Low	
4	CUST363	64	25	Low	
5	CUST454	46	25	Low	
6	CUST512	57	25	Low	
7	CUST791	51	25	Low	
8	CUST825	46	25	Low	

10.